



JOMO KENYATTA UNIVERSITY

OF

AGRICULTURE AND TECHNOLOGY (JKUAT)

SCHOOL OF ELECTRICAL, ELECTRONIC AND INFORMATION

ENGINEERING

DEPARTMENT OF TELECOMMUNICATION AND INFORMATION ENGINEERING

ICS 2403 DISTRIBUTED COMPUTING

ASSIGNMENT 1

CHRISPINE ODHIAMBO - ENE221-0132/2020

1. Introduction and Scope

A telecom operator runs network functions such as vRAN distributed units, user-plane functions, voice servers, video transcoders and IoT gateways on compute spread across small edge sites, which are close to users, and large core data centres, which are far away. Each function needs CPU, memory and interface bandwidth. If a function is hosted away from where its traffic enters the network, it also consumes transport bandwidth on every link in between and adds latency. Some functions cannot tolerate that latency: a DU needs about 2 ms.

Requests arrive and leave continuously, so allocation must be dynamic. Each request is admitted or blocked, placed, possibly resized while running, and released when its holding time ends. TeleRM S1 builds the foundation for this: nodes that own resources, processes that hold allocations, an allocator that decides where each process runs, enforcement of the allocation on the node, and the communication that ties these together. The guiding model is:

$$\text{System} = \text{Nodes} + \text{Processes} + \text{Resources} + \text{Communication}$$

2. Distributed OS versus Network OS

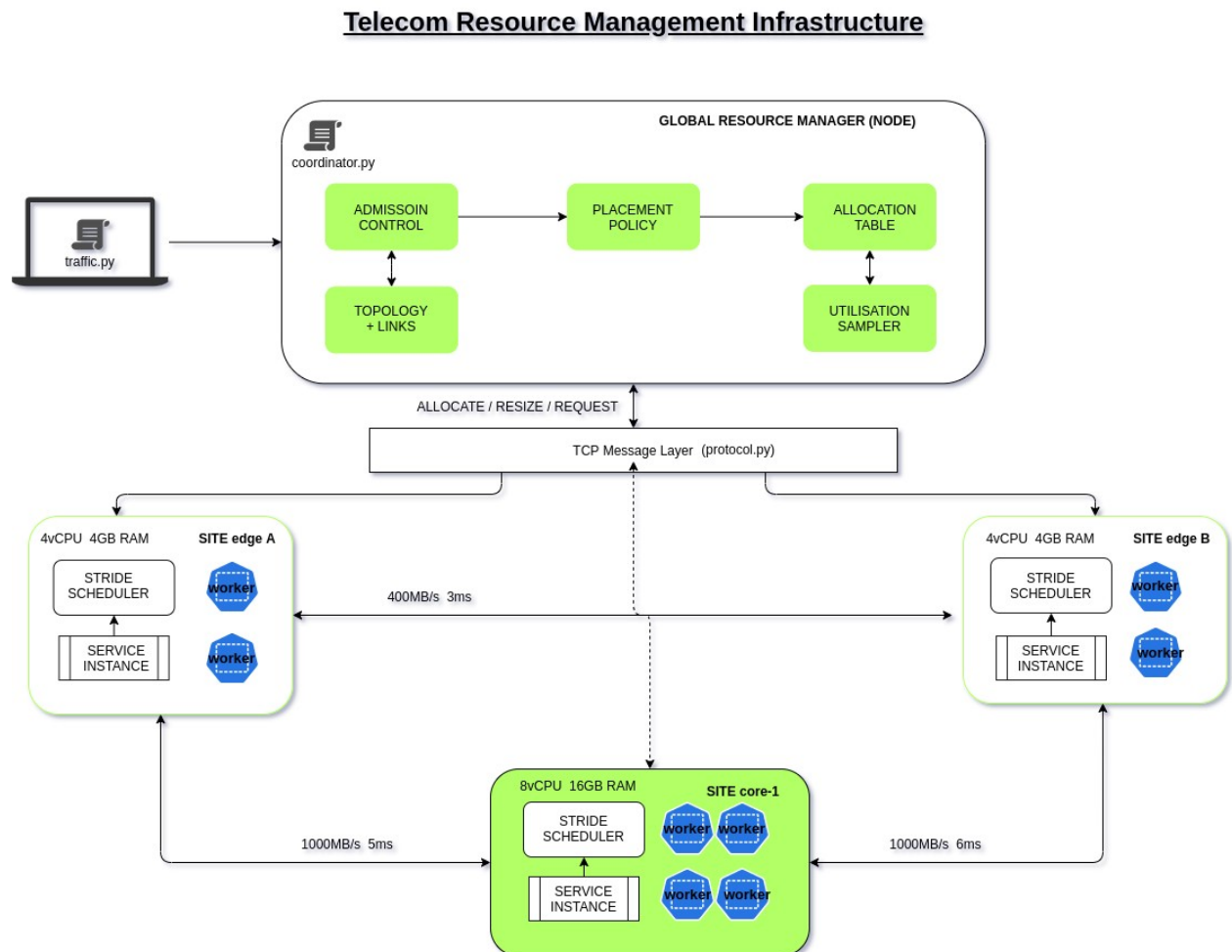
S1 is a middleware-based distributed OS layer running over ordinary host operating systems. It offers a single admission interface, global identifiers, and global resource and network accounting. Migration and shared state are deliberately out of scope; they arrive in Week 9 and Week 11. Table 1 shows where S1 sits relative to the two classic models.

Network OS vs distributed OS vs S1

Property	Network OS	Distributed OS	TeleRM S1
Where a function runs	Operator picks a server and deploys there	System decides	Resource manager decides from capacity, path bandwidth and latency
Resource view	Per server	Global	Global table of every site and every link
Identity	Local PIDs per server	System-wide	Global service-instance IDs (S00001, ...)
Enforcement	Local OS only	Coordinated	Decided globally, enforced locally by a stride scheduler
Node autonomy	Full	Low	Sites expire instances and schedule CPU on their own

3. System Architecture

A central Global Resource Manager holds the admission controller, placement policy, allocation table, topology with link reservations, site registry with failure detector, and a utilisation sampler. Three sites register with it and report back by heartbeat. A traffic generator or operator sends REQUEST, RESIZE and STATUS messages to the manager, which in turn sends ALLOCATE, RESIZE and RELEASE to the sites. Every node is a separate OS process with its own TCP port, so the system can run on one machine or several.



4. Process Model

Processes exist at two levels. The real level consists of OS processes: the manager, each site, and each site's worker pool. The logical level consists of service-instance processes. Each admitted request becomes an instance with a global ID, a resource allocation, a holding time and a state. While it runs it is backlogged, meaning it always has traffic-processing jobs waiting; the site's scheduler decides how often it gets a worker, and the jobs execute in the worker OS processes.

4.1 Instance record

Instance record fields

Field	Meaning
sid	Global instance ID from the manager
service, ingress	Service type; the site where its traffic enters
demand	Current allocation $d = (\text{cpu}, \text{mem}, \text{bw})$
holding_s, expires_at	Lifetime of the allocation
tickets, stride, pass_value	Stride-scheduling state (tickets = allocated millicores)
jobs_done, busy_s, latency_sum_ms	What the instance actually received

4.2 Life cycle and request handling

A request starts as Requested. It is Blocked if no site passes the capacity, bandwidth and latency checks; otherwise the node and path are reserved, ALLOCATE is sent and the instance becomes ADMITTED. The site then creates the instance and assigns tickets, moving it to RUNNING. A RESIZE updates tickets while remaining RUNNING. When the holding time ends, or on RELEASE, the instance is TERMINATED, an INSTANCE_EXIT is sent, and node and link resources are released.

End to end, the generator sends REQUEST(service, ingress, d , max latency, holding time). The manager checks every site for path latency, node capacity and link bandwidth. If none is feasible it replies BLOCKED with a reason. Otherwise it chooses a site by policy, reserves the node vector and path bandwidth, and sends ALLOCATE. The site creates the instance with tickets equal to $d.\text{cpu}$ and acknowledges; the manager replies ACCEPTED with site, path and latency. During the holding time the site dispatches jobs to workers in stride order, and on expiry it sends INSTANCE_EXIT with a summary so the manager can release the node and path.

5. Output

The output and results are located in the sub-directory milestone1/results